

GMUN Registration & Management Platform

Project Ideation & Technical Documentation

Project Type: Gitam Model United Nations Registration & Management Platform

Primary Objective: Provide a professional online registration, payment, and participant-management system for a Model United Nations event.

1. Project Overview

The proposed system is a web-based MUN registration platform designed to simplify the complete participant registration process.

It combines MUN information, online registration, committee/country preferences, online payment, payment verification, registration confirmation, an admin dashboard, participant management, search/filtering, Excel export, and optional email notifications.

2. Problem Statement

Traditional college-level MUN registration workflows often rely on disconnected forms, UPI payments, screenshots, spreadsheets, and separate communication channels.

Common issues include duplicate registrations, incorrect participant information, fake or incorrect payment screenshots, manual payment verification, difficulty tracking pending payments, data-entry errors, difficult reconciliation, and poor participant experience.

The proposed platform addresses these issues by connecting registration, payment, and participant management into one centralized workflow.

3. Proposed Solution

Registration → Participant details → Registration record → Payment → Payment verification → Registration confirmation → Database → Admin dashboard → Excel export.

4. Website Structure

Public side: Landing page, event information, committees, rules, registration, payment, and confirmation.

Admin side: Secure login, registration dashboard, participant records, payment status, search/filtering, statistics, and Excel export.

5. Landing Page

Suggested sections: Hero, About the MUN, event date, venue, theme, committees, registration fees, important dates, prizes, organizing team, FAQs, contact information, and a prominent Register Now CTA.

6. Registration System

Possible personal fields: Full Name, Email, Phone, College/University, Course, Year, and City.

Possible MUN fields: Participant Type, Committee Preference, Country Preference, Previous MUN Experience, and number of MUNs attended.

Optional fields may include T-shirt size, accommodation, food preferences, emergency contact, referral/source, and special requirements.

7. Registration Workflow

Participant → Website → Registration form → Server validation → Registration record → Payment initiation → Payment gateway → Payment completed → Server-side verification → Registration marked PAID → Confirmation → Optional email.

8. Registration ID

Every registration should receive a unique identifier such as MUN26-0001. The ID should be associated with the participant, registration, payment, and allocation information.

9. Payment System (The Below given 10 - 12 are the possible payment options.)

The payment layer should remain provider-independent. The initial implementation can support Razorpay, Cashfree, or Payment Links. The final provider should be selected based on merchant onboarding/KYC eligibility, settlement arrangements, applicable fees, and the organizer's preference.

10. Razorpay Integration

Backend creates a Razorpay order → Razorpay Checkout → participant pays → payment response → server-side verification → webhook confirmation → database update.

Important: The frontend must never be the sole source of truth for payment success. The backend should verify the payment and process provider webhooks.

11. Cashfree Integration

Backend creates a Cashfree order/payment session → Cashfree Checkout → participant pays → payment status is retrieved/received → webhook processed → database updated.

12. Payment Links

Registration → Registration ID → Payment Link → participant pays → payment notification/webhook → registration marked PAID.

Payment Links are useful as a fallback or simpler collection method, but for a larger event a direct checkout tied to a unique registration/order generally provides cleaner reconciliation.

13. Payment Abstraction

Registration System → Payment Service → Razorpay / Cashfree / Payment Links → Payment Status → Database.

The application should store normalized states such as PENDING, PAID, FAILED, REFUNDED, and CANCELLED, while keeping provider-specific IDs separately.

14. Database

Recommended database: Supabase PostgreSQL.

Registrations: Registration ID, name, email, phone, college, course, year, committee preference, country preference, participant type, registration date, and status.

Payments: Registration ID, provider, order ID, payment ID, amount, currency, status, payment date, payment method, and provider reference.

Admins: Admin ID, name, email, role, and created date.

15. Admin Dashboard

Dashboard statistics can include total registrations, paid registrations, pending payments, failed payments, revenue collected, registrations per day, committee-wise registrations, participant type breakdown, and college-wise registrations.

16. Registration Management

Admins should be able to view registrations, search by name/email/phone/registration ID, filter by committee/payment status/date, view complete participant information, and inspect payment details.

17. Excel Export

The admin can download MUN_Registrations_2026.xlsx.

Suggested columns: Registration ID, Registration Date, Full Name, Email, Phone, College, Course, Year, City, Participant Type, Committee, Country Preference, MUN Experience, Accommodation, T-Shirt Size, Amount, Payment Status, Payment Provider, Order ID, Payment ID, and Payment Date.

Optional worksheets: All Registrations, Paid Registrations, Pending Payments, Committee Summary, and Payment Summary.

18. Confirmation Page

After verified payment, show registration success, participant name, registration ID, committee, amount, and payment status. An email confirmation can also be sent.

19. Security Requirements

Validate inputs, verify payments server-side, verify webhook signatures, keep secret keys in environment variables, protect admin routes, enforce database authorization policies, prevent duplicate registrations, use HTTPS, and never store sensitive card information.

20. Recommended Technology Stack

Frontend: Next.js + React

UI: Tailwind CSS

Backend: Next.js server/API functions

Database: Supabase PostgreSQL

Payments: Razorpay / Cashfree / Payment Links

Authentication: Supabase Auth

Excel: SheetJS / XLSX

Hosting: Vercel

Repository: GitHub

21. High-Level Architecture

Participant → Next.js Website → Next.js Backend → Supabase PostgreSQL.

Payment branch: Next.js Backend ↔ Razorpay/Cashfree/Payment Links → webhook/status → Supabase.

Admin branch: Supabase → Admin Dashboard → Excel Export.

22. Deployment Architecture

GitHub → Vercel (which will be later redirected to the assigned domain “gmun.mdcgitam.in” → Next.js application/API → Supabase. The payment provider connects to backend APIs and webhooks through secure credentials.

23. Environment Variables

Examples: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET, or the equivalent Cashfree credentials.

will be deciding upon the decision of the payment method.

24. Client Responsibilities

MUN name, logo, theme, dates, venue, event description, committee list, registration fees, categories, rules, terms, refund policy, contact details, payment account/provider, required form fields, and communication content and the payment medium.

25. Developer Responsibilities

UI/UX implementation, website development, registration system, database, payment integration, verification/webhooks, admin dashboard, Excel export, deployment, testing, and agreed technical maintenance.

26. Payment Ownership Consideration

The production system will not be architected around the developer's personal bank account. The preferred structure is: organizing entity/approved merchant → payment gateway → designated bank account. The organizer should complete the provider's onboarding/KYC and maintain the merchant account.

27. Final MVP Scope

Public: Landing page, MUN information, committees, rules, fees, FAQs, registration, payment, confirmation.

Backend: PostgreSQL database, registration API, unique IDs, payment integration, verification, webhooks, and status management.

Admin: Secure login, dashboard, registration table, search, filters, payment status, registration details, and Excel export.

Payment: Architecture supporting Razorpay, Cashfree, or Payment Links.

28. Final Recommended Stack

Next.js + React + Tailwind CSS → Next.js Server/API → Supabase PostgreSQL → Razorpay/Cashfree/Payment Links → Supabase Auth + Admin Dashboard → XLSX/SheetJS → Vercel + GitHub.

29. Project Objective

The final product transforms the MUN registration process from disconnected forms, spreadsheets, and manual payment verification into a centralized digital platform.

Participant: Simple registration → secure payment → confirmation.

Organizer: Centralized data → verified payments → dashboard → Excel → better event management.

The architecture remains modular so the payment provider can be changed without rebuilding the entire application.

30. Architecture Summary

Layer	Technology	Purpose
Frontend	Next.js + React + Tailwind	Public website and registration UI
Backend	Next.js Server/API	Registration, payment and export APIs
Database	Supabase PostgreSQL	Registrations, payments and configuration
Payment	Razorpay / Cashfree / Payment Links	Online payment collection
Authentication	Supabase Auth	Admin access
Admin	Next.js dashboard	Manage registrations and payments
Export	XLSX / SheetJS	Download Excel reports
Hosting	Vercel + GoDaddy Domain	Application deployment